SQL

What is MySQL?

MySQL is a **database management system** used to **store, organize, and manage data**.
It uses a language called **SQL (Structured Query Language)** to interact with the data.
Example:
Apps like Facebook, Amazon, Instagram use databases to store user accounts, posts, orders, messages, etc.

What is a Database?

A **database** is like a digital storage system.
It stores information in **tables** (just like Excel sheets).
Example table:

id	**Name**	**age**
1	John	25
2	Priya	22

NUMERIC DATA TYPES

1) **INT**

Used for whole numbers (no decimals).
age INT;
Example: 10, 25, 100

2) DECIMAL (M, N)

Exact decimal values (perfect for money).
M = total digits
N = digits after decimal
Example:
salary DECIMAL(10,2);
Valid: 45000.50

STRING / TEXT DATA TYPES

1)CHAR(n)

Fixed length string.
gender CHAR(1);
Example:
Example: 'M', 'F'

2)VARCHAR(n)

Variable length string.
Most commonly used.
name VARCHAR(50);
Example:
'John', 'Priya'

3)VARCHAR(n)

Variable length string.
Most commonly used.
Example:
name VARCHAR(50);
Example: 'John', 'Priya'

4)ENUM

Stores one value from a list.
Example:
status ENUM('active', 'inactive', 'pending')

5)BOOLEAN / BOOL

Stores TRUE or FALSE.
Example:
Isactive BOOLEAN;

```
CREATE TABLE employees (
   empid INT PRIMARY KEY,
   name VARCHAR(100),
   age INT,
   salary DECIMAL(10,2),
   isactive BOOLEAN,
   status ENUM('active', 'inactive', 'onleave')
);
```

Sample data

INSERT INTO employees (empid,name,age,salary,isactive,status) VALUES
(1, 'Rahul Sharma', 28, 35000.00, TRUE, 'active');

CREATE DATABASE----------→CREATE TABLE ----------→INSERT DATA INTO TABLE

TASK1:

CREATE TABLE FOR WORKOUT THE CLAUSE

```
CREATE TABLE employees (
    empid INT,
    name VARCHAR(50),
    department VARCHAR(20),
    age INT,
    salary INT
);
```

INSERT SAMPLE DATA

```
INSERT INTO employees VALUES
(1, 'Rahul',  'HR',       28, 30000),
(2, 'Priya',  'Finance',  32, 50000),
(3, 'Kumar',  'HR',       26, 28000),
(4, 'John',   'IT',       30, 45000),
(5, 'Neha',   'IT',       29, 52000),
(6, 'Asha',   'Finance',  35, 47000),
(7, 'Vijay',  'IT',       40, 60000),
(8, 'Rita',   'HR',       31, 35000);
```

MySQL Clauses

1)SELECT Clause

Used to choose which columns you want to see.

Example

SELECT * FROM employees;

(Fetches all columns from the products table.)

empid	name	dept	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	32	50000
3	Kumar	HR	26	28000
4	John	IT	30	45000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000
8	Rita	HR	31	35000

Used to select specific columns.

SELECT name, salary FROM employees;

(Shows only name and salary columns.)

name	***salary***
Rahul	*30000*
Priya	*50000*
Kumar	*28000*
John	*45000*
Neha	*52000*
Asha	*47000*
Vijay	*60000*
Rita	*35000*

2)**WHERE Clause**

Filters rows based on a **condition**.

Example:

```
SELECT * FROM employees
WHERE department = 'IT';
```

Shows only employees who work in **IT** department

emp_id	name	department	age	salary
4	John	IT	30	45000
5	Neha	IT	29	52000
7	Vijay	IT	40	60000

3)ORDER BY CLAUSE

Sorts results in **ascending (ASC)** or **descending (DESC)** order.

Example:

```
SELECT name, salary FROM employees
ORDER BY salary DESC;
```

name	salary
Vijay	60000
Neha	52000
Priya	50000
Asha	47000
John	45000
Rita	35000
Rahul	30000
Kumar	28000

4) GROUP BY Clause

Groups rows having the same value in a column.

EXAMPLE:

```
SELECT department FROM employees GROUP BY department;
```

department
Finance
HR
IT

5) **HAVING Clause**

Filters groups created by **GROUP BY**.

(WHERE cannot be used after grouping)

EXAMPLE:

```
SELECT department, COUNT(*) AS total
FROM employees
GROUP BY department
HAVING total <10;
```

Department	**Count**
HR	3
Finance	2
IT	3

6) **LIMIT Clause**

Limits the number of rows returned.

EXAMPLE:

```
SELECT * FROM employees
LIMIT 3;
```

Shows only **first 3** orders.

empid	name	department	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	32	50000
3	Kumar	HR	26	28000

7) **AS Clause (Alias)**

Used to give a temporary name to a column or table.

Example:

```
SELECT name AS employee_name, salary AS monthly_salary
FROM employees;
```

employee_name	monthly_salary
Rahul	30000
Priya	50000
Kumar	28000
John	45000
Neha	52000
Asha	47000
Vijay	60000
Rita	35000

8) **DISTINCT Clause**

Removes duplicate values.

Example:

SELECT DISTINCT department FROM employees;

Shows each department name **only once**.

department
HR
Finance
IT

9)IN Clause

Match values from a list.

Example

```
SELECT name, department
FROM employees
WHERE department IN ('HR', 'Finance');
```

name	department
Rahul	HR
Kumar	HR
Rita	HR
Priya	Finance
Asha	Finance

10) **BETWEEN Clause**

Values in a range.

Query:

```
SELECT name, salary
FROM employees
WHERE salary BETWEEN 30000 AND 50000;
```

name	salary
Rahul	30000
John	45000
Priya	50000
Asha	47000
Rita	35000

11). LIKE Clause

Pattern matching.

Query:

```
SELECT * FROM employees
WHERE name LIKE 'R%';
```

empid	name	department	age	salary
1	Rahul	HR	28	30000
8	Rita	HR	31	35000

LOGICAL OPERATORS:

1) AND
2) OR
3) NOT

1)AND

Used when **both conditions must be true**.

EXAMPLE:

```
SELECT * FROM employees
WHERE age > 25 AND salary > 40000;
```

empid	name	department	age	salary
2	Priya	Finance	32	50000
4	John	IT	30	45000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000

2)**OR Operator**

Used when **at least one condition is true**.

Query:

```
SELECT * FROM employees
WHERE age < 28 OR salary > 50000;
```

empid	name	department	age	salary
3	**Kumar**	**HR**	**26**	**28000**
5	**Neha**	**IT**	**29**	**52000**
7	**Vijay**	**IT**	**40**	**60000**

3)**NOT Operator**

Used to **negate a condition**.

Query:

```
SELECT * FROM employees
WHERE NOT department = 'HR';
```

empid	name	dept	age	salary
2	Priya	Finance	32	50000
4	John	IT	30	45000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000

Combination of AND + OR

Select employees from HR OR (age > 30 AND salary > 40000)

EXAMPLE:

```
SELECT * FROM employees
WHERE department = 'HR' OR (age > 30 AND salary > 40000);
```

empid	name	dept	age	salary
1	Rahul	HR	28	30000
3	Kumar	HR	26	28000
8	Rita	HR	31	35000
2	Priya	Finance	32	50000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000

AGGREGATE FUNCTION:

Function	Meaning
COUNT()	Count rows
SUM()	Total
AVG()	Average
MIN()	Smallest value
MAX()	Largest value
COUNT(DISTINCT)	Count unique rows
SUM(DISTINCT)	Sum unique
AVG(DISTINCT)	Avg unique

1)COUNT()

What it does:

Counts the number of rows in the table (or in a group)

Query:

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

Explanation:

- COUNT(*) counts ALL rows.
- Your table has **8 rows**.

total_employees
8

2)SUM()

What it does:

Adds all numeric values of a column.

Query:

```
SELECT SUM(salary) AS total_salary
FROM employees;
```

Explanation:

It adds all salaries:

30000 + 50000 + 28000 + 45000 + 52000 + 47000 + 60000 + 35000 = **347000**

total_salary
347000

3) AVG()

What it does:

Finds the **average value** of a numeric column.

Query:

```
SELECT AVG(salary) AS avg_salary
FROM employees;
```

Explanation:

Average = Total Salary ÷ Number of Employees

= 347000 ÷ 8

= **43375**

avg_salary
43375

4)MAX()

What it does:

Returns the **largest value**.

Query:

```
SELECT MAX(salary) AS highest_salary
FROM employees;
```

Explanation:

Highest salary = **60000 (Vijay)**

highest_salary
60000

5) MIN()

What it does:

Returns the **smallest value**.

Query:

```
SELECT MIN(salary) AS lowest_salary
FROM employees;
```

Explanation:

Lowest salary = **28000 (Kumar)**

lowest_salary
28000

GROUPING:

GROUP BY groups rows having the same value into **one group**

QUERY:

```
SELECT department, COUNT(*) AS total_emp
FROM employees
GROUP BY department;
```

1.All HR employees become 1 group
2.All IT employees become 1 group
3.All Finance employees become 1 group

dept	total_emp
HR	3
Finance	2
IT	3

HR → 3 employees
Finance → 2 employees
IT → 3 employees

HAVING VS WHERE CLAUSE

WHERE	HAVING
Filters rows before grouping	Filters groups after grouping
Cannot use aggregate functions (SUM, COUNT, AVG…)	Can use aggregate functions
Used with any SELECT query	Only works with GROUP BY
Applies to individual rows	Applies to group results

1)WHERE + GROUP BY (Allowed)

Query:

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
WHERE age > 30
GROUP BY department;
```

Explanation:

WHERE filters rows where age > 3
Then GROUP BY groups the remaining rows by department.

Step-by-step:

WHERE age > 30 → Keep only these rows:

empid	name	dept	age	salary
2	Priya	Finance	32	50000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000
8	Rita	HR	31	35000

Now group by dept:

- Finance: (50000 + 47000) / 2 = 48500
- IT: (60000) / 1 = 60000
- HR: (35000) / 1 = 35000

FINAL OUTPUT:

dept	Avg salary
Finance	48500.00
IT	60000.00
HR	35000.00

2)HAVING

QUERY:

```
SELECT department, COUNT(*) AS total_emp
FROM employees
GROUP BY department
HAVING COUNT(*) > =2;
```

STEP 1 — Group the employees by department

1) HR Group

empid	name	dept	age	salary
1	Rahul	HR	28	30000
3	Kumar	HR	26	28000
8	Rita	HR	31	35000

2) Finance Group

empid	name	dept	age	salary
2	Priya	Finance	32	50000
6	Asha	Finance	35	47000

3) IT Group

empid	name	dept	age	salary
4	John	IT	30	45000
5	Neha	IT	29	52000
7	Vijay	IT	40	60000

STEP 2 — Final GROUP BY Output Table

dept	total_emp
HR	3
Finance	2
IT	3

SET OPERATORS:

UNION VS UNIONALL:

Table 1: employees_2023

empid	name	dept
1	Rahul	HR
2	Priya	Finance
3	Kumar	HR

Table 2: employees_2024

empid	name	dept
3	Kumar	HR
4	John	IT
5	Neha	IT

UNION:

Removes duplicates
Combines results from both queries

QUERY:

```
SELECT name, dept FROM employees_2023
UNION
SELECT name, dept FROM employees_2024;
```

name	dept
Rahul	HR
Priya	Finance
Kumar	HR
John	IT
Neha	IT

UNION ALL

- Does NOT remove duplicates
- Faster than UNION
- Includes all rows

QUERY:

```
SELECT name, dept FROM employees_2023
UNION ALL
SELECT name, dept FROM employees_2024;
```

name	dept
Rahul	HR
Priya	Finance
Kumar	HR
Kumar	HR
John	IT
Neha	IT

UNION	UNION ALL	
Removes duplicate rows	**Yes**	**No**
unique rows	**Yes**	**No**
Speed	**Slow (because it removes duplicates)**	**Fast**
When to use?	**When duplicates not allowed**	**When duplicates allowed**

ANY & ALL

They are used with subqueries to compare a value with a set of values.

1)ANY

Condition is **TRUE if ANY ONE** value in the list matches the condition.

Even if **1 value** satisfies, it becomes TRUE.

Sample Data:

empid	name	dept	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	32	50000
3	Kumar	HR	26	28000
4	John	IT	30	45000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000
8	Rita	HR	31	35000

SUBQUERY:

SELECT salary FROM employees WHERE dept = 'IT';

salary
45000
52000
60000

QUERY:

```
SELECT name, salary
FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE dept = 'IT');
```

ANY :

salary > ANY (45000, 52000, 60000)
ANY = At least one

This becomes:

salary > 45000

OR

salary > 52000

OR

salary > 60000

Even if **one** is true → accept the row.

name	salary	Is salary > ANY IT salary?
Rahul	30000	**30000 > 45000** ,30000>52000,30000>60000, (1ST CONDITION SATISFY)
Priya	50000	**50000 > 45000** ,50000>52000,50000>60000 (1ST CONDITION SATISFY)
Kumar	28000	28000 > 45000,28000>52000,28000>60000 (NO CONDITION SATISFY)
John	45000	45000 > 45000,45000>52000,45000>60000 (NO CONDITION SATISFY)
Neha	52000	**52000 > 45000**,52000>52000,52000>60000 (1ST CONDITION SATISFY)
Asha	47000	**47000 > 45000**, 47000>52000,47000>60000 (1ST CONDITION SATISFY)
Vijay	60000	**60000 > 45000,60000>52000**,60000>60000 (1ST ,2ND CONDITION SATISFY)

name	salary	Is salary > ANY IT salary?
Rita	35000	35000 > 45000,35000>52000,35000>60000 (NO CONDITION SATISFY)

OUTPUT:

name	salary
Priya	50000
Neha	52000
Asha	47000
Vijay	60000

2) ALL

Condition is **TRUE only if ALL values** satisfy the condition.

Every value must satisfy the condition.

QUERY

```
SELECT name, salary
FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE dept = 'IT');
```

ALL :

salary > ALL (45000, 52000, 60000)

This becomes:

salary > 45000

AND

salary > 52000

AND

salary > 60000

salary > 45000
salary > 52000
salary > 60000

All must be TRUE → salary must be greater than 60000.
STEP-BY-STEP CHECK

name	salary	Is salary > ALL IT salary?
Rahul	30000	**30000 > 45000 ,30000>52000,30000>60000, (NO CONDITION SATISFY)**
Priya	50000	**50000 > 45000 ,50000>52000,50000>60000 (NO CONDITION SATISFY)**
Kumar	28000	**28000 > 45000,28000>52000,28000>60000 (NO CONDITION SATISFY)**
John	45000	**45000 > 45000,45000>52000,45000>60000 (NO CONDITION SATISFY)**
Neha	52000	**52000 > 45000,52000>52000,52000>60000 (NO CONDITION SATISFY)**
Asha	47000	**47000 > 45000, 47000>52000,47000>60000 (NO CONDITION SATISFY)**
Vijay	60000	**60000 > 45000,60000>52000,60000>60000 (NO CONDITION SATISFY)**
Rita	35000	**35000 > 45000,35000>52000,35000>60000 (NO CONDITION SATISFY)**

OUTPUT:

NO ROW

(There is no employee with salary > 60000, so ALL returns an EMPTY result.)

Feature	ANY	ALL
Meaning	**Condition must be TRUE for at least one value**	**Condition must be TRUE for every value**
Logical type	**Works like OR**	**Works like AND**
Condition success	**Even one match makes it TRUE**	**All values must match**
Allowed	**With subqueries**	**With subqueries**
Equivalent to	**> ANY → compare with minimum value**	**> ALL → compare with maximum value**

SAMPLE DATA:

empid	name	dept	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	32	50000
3	Kumar	HR	26	28000
4	John	IT	30	45000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000
8	Rita	HR	31	35000

WILDCARD LIKE:

% → any number of characters
_ → one single character

1) Starts with "R"

```
SELECT * FROM employees
WHERE name LIKE 'R%';
```

OUTPUT:

empid	name	dept	age	salary
1	Rahul	HR	28	30000
8	Rita	HR	31	35000

2) Names ending with a

```
SELECT * FROM employees
WHERE name LIKE '%a';
```

OUTPUT:

empid	name	dept	age	salary
2	Priya	Finance	32	50000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000
8	Rita	HR	31	35000

3)Names containing 'a' anywhere

```
SELECT * FROM employees
WHERE name LIKE '%a%';
```

OUTPUT

empid	name	dept	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	32	50000
3	Kumar	HR	26	28000
5	Neha	IT	29	52000
7	Vijay	IT	40	60000

empid	name	dept	age	salary
8	Rita	HR	31	35000

4) **Names containing 'ha'**

```
SELECT * FROM employees
WHERE name LIKE '%ha%';
```

OUTPUT:

empid	name	dept	age	salary
5	Neha	IT	29	52000
6	Asha	Finance	35	47000

5) **Second letter is a**

```
SELECT * FROM employees
WHERE name LIKE '_a%';
```

OUTPUT

empid	name	dept	age	salary
1	Rahul	HR	28	30000

5) **Names with exactly 5 letters**

```
SELECT * FROM employees
WHERE name LIKE '_____';
```

OUTPUT:

empid	name	dept	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	32	50000
3	Kumar	HR	26	28000
7	Vijay	IT	40	60000

6) **Third letter is h**

```
SELECT * FROM employees
WHERE name LIKE '__h%';
```

OUTPUT

empid	name	dept	age	salary
1	Rahul	HR	28	30000
4	John	IT	30	45000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000

7) **Name starts with R and ends with a**

```
SELECT * FROM employees
WHERE name LIKE 'R%a';
```

OUTPUT:

empid	name	dept	age	salary
8	Rita	HR	31	35000

8) **Second letter "i" and ends with "a"**

```
SELECT * FROM employees
WHERE name LIKE '_i%a';
```

OUTPUT:

empid	name	dept	age	salary
8	Rita	HR	31	35000

9) **Names ending with "ha"**

```
SELECT * FROM employees
WHERE name LIKE '%ha';
```

OUTPUT:

empid	name	dept	age	salary
5	Neha	IT	29	52000
6	Asha	Finance	35	47000

NOT LIKE :

1) **Names NOT starting with R**

```
SELECT * FROM employees
WHERE name NOT LIKE 'R%';
```

OUTPUT

empid	name	dept	age	salary
2	Priya	Finance	32	50000
3	Kumar	HR	26	28000
4	John	IT	30	45000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000

2) Names NOT containing "a"

```
SELECT * FROM employees
WHERE name NOT LIKE '%a%';
```

OUTPUT

empid	name	dept	age	salary
4	John	IT	30	45000

NULL

NULL is simply **nothing.**

SAMPLE TABLE:

```
CREATE TABLE employee_info (
   empid INT,
   name VARCHAR(50),
   department VARCHAR(30),
   age INT,
   salary INT
);
```

INSERT DATA:

```
INSERT INTO employee_info VALUES
(1, 'Rahul', 'HR', 28, 30000),
(2, 'Priya', 'Finance', NULL, 50000),
(3, NULL, 'HR', 26, 28000),
(4, 'John', NULL, 30, 45000),
(5, 'Neha', 'IT', NULL, NULL),
(NULL, 'Asha', 'Finance', 35, 47000),
(7, NULL, NULL, NULL, NULL);
```

AFTER INSERT TABLE LOOKS LIKE:

empid	name	department	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	NULL	50000
3	NULL	HR	26	28000
4	John	NULL	30	45000
5	Neha	IT	NULL	NULL
NULL	Asha	Finance	35	47000
7	NULL	NULL	NULL	NULL

1) **Find employees whose SALARY is NULL**

```
SELECT * FROM employee_info
WHERE salary IS NULL;
```

NOTE:*IS NULL checks where salary has no value.*

OUTPUT:

empid	name	department	age	salary
5	Neha	IT	NULL	NULL
7	NULL	NULL	NULL	NULL

2) **Find employees where DEPARTMENT is NOT NULL**

```
SELECT * FROM employee_info
WHERE department IS NOT NULL;
```

NOTE: ***Shows only rows where department has a value.***

OUTPUT:

empid	name	department	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	NULL	50000
3	NULL	HR	26	28000
5	Neha	IT	NULL	NULL
NULL	Asha	Finance	35	47000

3) Count how many NULL names exist

```
SELECT COUNT(*) AS null_names
FROM employee_info
WHERE name IS NULL;
```

NOTE: ***Counts the number of rows where name is missing.***

OUTPUT:

null_names
2

4) Display rows where ANY field is NULL

```
SELECT *
FROM employee_info
WHERE empid IS NULL
  OR name IS NULL
  OR department IS NULL
  OR age IS NULL
  OR salary IS NULL;
```

NOTE: ***Shows all rows that contain at least one NULL value***

OUTPUT:

empid	name	department	age	salary
2	Priya	Finance	NULL	50000
3	NULL	HR	26	28000
4	John	NULL	30	45000
5	Neha	IT	NULL	NULL
NULL	Asha	Finance	35	47000
7	NULL	NULL	NULL	NULL

5) **Display only rows that have NO NULL values**

```
SELECT *
FROM employee_info
WHERE empid IS NOT NULL
  AND name IS NOT NULL
  AND department IS NOT NULL
  AND age IS NOT NULL
  AND salary IS NOT NULL;
```

This query shows only those records where:

- empid is not NULL
- name is not NULL
- department is not NULL
- age is not NULL
- salary is not NULL

All 5 fields must contain values

If even **one field contains NULL**, the row will NOT appear

OUTPUT:

empid	name	department	age	salary
1	Rahul	HR	28	30000

Only Rahul's row has no NULL values.
So it is the only row displayed.

Condition	Meaning	Returns Rows Where…
IS NULL	Checks if a value is NULL	The column contains NULL value
IS NOT NULL	Checks if a value is NOT NULL	The column contains any value except NULL

1)IFNULL:

1) **Replace NULL salary with 0 (using IFNULL)**

```
SELECT empid, name, IFNULL(salary, 0) AS updated_salary
FROM employee_info;
```

NOTE*: IFNULL(column_name, value) → replaces NULL with value.*

OUTPUT:

empid	name	updated_salary
1	Rahul	30000
2	Priya	50000
3	NULL	28000
4	John	45000
5	Neha	0
NULL	Asha	47000
7	NULL	0

2) **COALESCE(value1, value2, value3, …)**

- Returns the first NON-NULL value
- Works with multiple columns
- More powerful than IFNULL

QUERY:

```
SELECT empid,
COALESCE(name, department, 'No Data') AS employee_name
FROM employee_info;
```

OUTPUT:

empid	employee_name
1	Rahul
2	Priya
3	HR
4	John
5	Neha
NULL	Asha
7	No Data

Explanation:

- COALESCE picks the first non-NULL value
  Example:
- For empid 3 → name is NULL → takes department 'HR'
- For empid 7 → both name and dept are NULL → shows 'No Data'

3) NULLIF()

NULLIF(a, b)

- If a = b, then result = NULL
- If a ≠ b, then result = a

QUERY:

```
SELECT name, age, NULLIF(age, 30) AS result
FROM employee_info;
```

SAMPLE DATA:

empid	name	department	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	NULL	50000
3	NULL	HR	26	28000
4	John	NULL	30	45000
5	Neha	IT	NULL	NULL
NULL	Asha	Finance	35	47000
7	NULL	NULL	NULL	NULL

OUTPUT:

name	**age**	**Compare age = 30?(REFERENCE ONLY NOT DISPLAY IN OUTPUT)**
Rahul	28	No
Priya	NULL	NULL = 30? No
NULL	26	No
John	NULL	Yes
Neha	NULL	No
Asha	35	No
NULL	NULL	No

UPDATE:

1)UPDATE one column

Task: Increase Rahul's salary to 35000

```
UPDATE employee_info
SET salary = 35000
WHERE name = 'Rahul';
```

OUTPUT:

empid	name	salary
1	Rahul	35000

2)UPDATE multiple columns

Task: For Priya → set age = 33 and department = 'HR'

```
UPDATE employee_info
SET age = 33,
Department = 'HR'
WHERE name = 'Priya';
```

OUTPUT:

empid	name	department	age
2	Priya	HR	33

3)UPDATE salary by Percentage Particular department:

Task: Increase Salary of IT Employees by 10%

```
UPDATE employees
SET salary = salary * 1.10
WHERE dept = 'IT';
```

(HINT:-10/100=1.10)

OUTPUT:

name	dept	old salary(NOT DISPLAY IN OUTPUT)	new salary
John	IT	45000	49500
Neha	IT	52000	57200
Vijay	IT	60000	66000

4) **UPDATE using empid**

Task: Change Rita's age to 33

```
UPDATE employees
SET age = 33
WHERE empid = 8;
```

OUTPUT:

empid	name	age
8	Rita	33

5) **UPDATE with a Condition (age < 30)**

Task: Add 2000 bonus to employees age < 30

```
UPDATE employees
SET salary = salary + 2000
WHERE age < 30;
```

OUTPUT:

name	age	old salary(reference only not show in result)	new salary
Rahul	28	35000	**37000**
Kumar	26	28000	**30000**
Neha	29	57200	**59200**

6) UPDATE All Rows (WARNING)

Task: Set department to "General" for ALL employees

```
UPDATE employees
SET dept = 'General';
```

NOTE: WITHOUT THE WHERE CLASS IT UPDATE THE ALL DATAS IN COLUMN

OUTPUT:

empid	name	dept
1	Rahul	General
2	Priya	General
3	Kumar	General
4	John	General
5	Neha	General
6	Asha	General
7	Vijay	General
8	Rita	General

DELETE:

SAMPLE DATA:

empid	name	dept	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	32	50000
3	Kumar	HR	26	28000
4	John	IT	30	45000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000
8	Rita	HR	31	35000

1)DELETE a specific row

```
DELETE FROM employees
WHERE empid = 4;
```

OUTPUT:

empid	name	dept
1	Rahul	HR
2	Priya	Finance
3	Kumar	HR
5	Neha	IT
6	Asha	Finance
7	Vijay	IT
8	Rita	HR

John removed

2)DELETE multiple rows based on a condition

Task: Delete all employees from HR department

```
DELETE FROM employees
WHERE dept = 'HR';
```

OUTPUT:

empid	name	dept
2	Priya	Finance
5	Neha	IT
6	Asha	Finance
7	Vijay	IT

3)DELETE rows with salary LESS than 40000

```
DELETE FROM employees
WHERE salary < 40000;
```

OUTPUT

empid	name	salary
2	Priya	50000
5	Neha	52000
6	Asha	47000
7	Vijay	60000

Rows with salary 30000, 28000, 35000 removed

4)DELETE ALL ROWS (WARNING)

Removes all data from the table (table becomes empty)

```
DELETE FROM employees;
```

OUTPUT:

empid	name	dept	age	salary
(no rows)				

- **Table is now empty**
- **Table structure remains**

5)DELETE using IS NULL

If any NULL values exist:

```
DELETE FROM employees
WHERE salary IS NULL;
```

Removes all rows where salary is NULL.

6)DELETE using LIMIT

Delete only one row (first matched row)

```
DELETE FROM employees
WHERE dept = 'IT'
LIMIT 1;
```

Only one IT employee will be deleted
(not all)

7)DELETE using AND condition

Task: Delete IT employees who earn more than 55000

Query:

```
DELETE FROM employees
WHERE dept = 'IT'
AND salary > 55000;
```

Output:

empid	name	dept	salary
5	Neha	IT	52000
4	John	IT	45000

Vijay removed (60000 > 55000)

AUTO_INCREMENT

What is AUTO_INCREMENT?

AUTO_INCREMENT automatically gives the next number to a column

when you insert a new row.

- Used mostly for ID or primary key
- You don't need to manually give a value
- MySQL increases the value automatically

STEP 1 — Create a table with AUTO_INCREMENT

```
CREATE TABLE student (
   id INT AUTO_INCREMENT,
   name VARCHAR(20),
   PRIMARY KEY (id)
);
```

Now id will generate numbers automatically.

STEP 2 — Insert data without giving id

```
INSERT INTO student (name) VALUES ('Rahul');
INSERT INTO student (name) VALUES ('Priya');
INSERT INTO student (name) VALUES ('John');
```

OUTPUT

id	name
1	Rahul
2	Priya
3	John

MySQL automatically filled 1, 2, 3 for id.

STEP 4 — Insert another row

```
INSERT INTO student (name) VALUES ('Neha');
```

OUTPUT:

id	name
4	Neha

STEP-5: AUTO_INCREMENT WORKS After DELETE

DELETE FROM student WHERE id = 3;

OUTPUT:

id	name
1	Rahul
2	Priya
4	Neha

STEP-6: Now insert a new row:

INSERT INTO student (name) VALUES ('Asha');

OUTPUT:

id	name
1	Rahul
2	Priya
4	Neha
5	Asha

MySQL does NOT reuse deleted numbers (id = 3).
Next id continues → 5

1)AUTO_INCREMENT WORKS After TRUNCATE

TRUNCATE TABLE student;

Table becomes empty.TRUNCATE TABLE **cannot be used with a WHERE clause** in MySQL.

Insert again:

INSERT INTO student (name) VALUES ('Kumar');

OUTPUT:

id	name
1	Kumar

TRUNCATE resets AUTO_INCREMENT to 1

AUTO_INCREMENT RULES:

1) Only one AUTO_INCREMENT per table
2) Must be an integer type
3) Usually used with PRIMARY KEY

What is CASE in MySQL?

CASE = **IF / ELSE IF** inside SQL.
It allows you to **create new columns** based on different conditions.

SYNTAX:

```
CASE
   WHEN condition THEN value
   WHEN condition THEN value
   ELSE value
END
```

This creates a new **calculated column**.

SAMPLE DATA

empid	name	dept	age	salary
1	Rahul	HR	28	30000
2	Priya	Finance	32	50000
3	Kumar	HR	26	28000
4	John	IT	30	45000
5	Neha	IT	29	52000
6	Asha	Finance	35	47000
7	Vijay	IT	40	60000
8	Rita	HR	31	35000

1)CASE Example — Salary Category

```
SELECT name, salary,
CASE
   WHEN salary >= 50000 THEN 'High Salary'
   WHEN salary >= 30000 THEN 'Medium Salary'
   ELSE 'Low Salary'
END AS salary_level
FROM employees;
```

OUTPUT:

name	salary	salary_level
Rahul	30000	Medium Salary
Priya	50000	High Salary
Kumar	28000	Low Salary
John	45000	Medium Salary
Neha	52000	High Salary
Asha	47000	Medium Salary
Vijay	60000	High Salary
Rita	35000	Medium Salary

2)CASE Example — Age Group

```
SELECT name, age,
CASE
   WHEN age < 30 THEN 'Young'
   WHEN age BETWEEN 30 AND 40 THEN 'Middle Age'
   ELSE 'Senior'
END AS age_group
FROM employees;
```

Output:

name	age	age_group
Rahul	28	Young
Priya	32	Middle Age
Kumar	26	Young
John	30	Middle Age
Neha	29	Young
Asha	35	Middle Age
Vijay	40	Middle Age
Rita	31	Middle Age

3)CASE Example — Bonus Based on Department:

```
SELECT name, dept,
CASE
        WHEN dept = 'IT' THEN 5000
        WHEN dept = 'HR' THEN 3000
         ELSE 2000
END AS bonus
FROM employees;
```

OUTPUT:

name	dept	bonus
Rahul	HR	3000
Priya	Finance	2000
Kumar	HR	3000
John	IT	5000
Neha	IT	5000
Asha	Finance	2000
Vijay	IT	5000
Rita	HR	3000

4)CASE in ORDER BY (Custom Sorting)

We want to sort departments in this order:

1. IT (high priority)
2. HR
3. Finance (lowest)

QUERY:

```
SELECT name, dept
FROM employees
ORDER BY
CASE
        WHEN dept = 'IT' THEN 1
        WHEN dept = 'HR' THEN 2
ELSE 3
END;
```

CASE assigns numbers:

- IT → 1
- HR → 2
- Finance → 3

MySQL sorts by these numbers.

OUTPUT

name	**dept**
John	IT
Neha	IT
Vijay	IT
Rahul	HR
Kumar	HR
Rita	HR
Priya	Finance
Asha	Finance

COMMENTS in MySQL

Comments are used to add notes inside SQL code.
They help explain what the query does.

MySQL supports two types of comments:

1) **Single Line Comment using –**
2) **Multi-line Comment /* ... */**

1)Single Line Comment using –

```
-- This deletes HR department employees
DELETE FROM employees
WHERE dept = 'HR';
```

Explain:

MySQL ignores the comment
Reads only the DELETE query

i)Inline Comment

You can also write comments after a SQL statement.

```
SELECT * FROM employees; -- Display all employee records
```

2)Multi-line comment:

```
/*
This query shows employees
who have salary more than 40000
*/
SELECT * FROM employees
WHERE salary > 40000;
```

Explain:

Used for long explanations or multiple lines.

What is EXISTS in MySQL?

EXISTS is used in a subquery to check if at least one row exists.

If the subquery returns 1 or more rows → EXISTS = TRUE
If subquery returns 0 rows → EXISTS = FALSE

SIMPLE EXISTS Example

`departments

dept_id	dept_name
1	HR
2	Finance
3	IT

`employees

empid	name	dept_name
1	Rahul	HR
2	Priya	Finance
3	John	IT
4	Kumar	HR

GOAL:

Show all departments **that have at least one employee**.

```
SELECT dept_name
        FROM departments d
        WHERE EXISTS (
                 SELECT *
                FROM employees e
                WHERE e.dept_name = d.dept_name
                );
```

We check each department one by one.

Step 1: **Check department HR**

Subquery becomes:

```
SELECT *
FROM employees
WHERE dept_name = 'HR';
```

Employees in HR:

- Rahul
- Kumar

Subquery returns rows → EXISTS = TRUE
HR is included.

Step 2: Check department Finance

Subquery:

```
SELECT *
FROM employees
WHERE dept_name = 'Finance';
```

Employees in Finance:

- Priya

Subquery returns 1 row → EXISTS = TRUE
Finance is included.

Step 3: Check department IT

Subquery:

```
WHERE dept_name = 'IT';
```

Employees in IT:

- John

Subquery returns 1 row → EXISTS = TRUE
IT is included.

Final Output:

dept_name
HR
Finance
IT

All departments have at least one employee — so all appear

NOT EXISTS

Show departments that **do NOT have any employees**.

```
SELECT dept_name
FROM departments d
WHERE NOT EXISTS (
SELECT *
FROM employees e
WHERE e.dept_name = d.dept_name
);
```

Since all departments have employees:

Output:

(No rows)

Operator	Meaning
EXISTS	TRUE if subquery returns ≥ 1 row
NOT EXISTS	TRUE if subquery returns 0 rows

JOINS

INNER JOINS

INNER JOIN returns only those rows where the join condition is true in both tables. Non-matching rows are excluded.

empid	name	dept_id
1	Rahul	1
2	Priya	2
3	John	3
4	Neha	5

dept_id	dept_name
1	HR
2	Finance
3	IT
4	Admin

QUERY:

```
SELECT e.empid, e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

employees e → left table (alias e)
departments d → right table (alias d)
ON e.dept_id = d.dept_id → joining condition (match dept ids)
INNER JOIN = only rows where condition is TRUE in both tables

RESULT:

empid	name	dept_name
1	Rahul	HR
2	Priya	Finance
3	John	IT

RIGHT JOIN:

RIGHT JOIN returns:

All rows from the RIGHT table
Matching rows from the LEFT table
If no match is found → NULL comes from the LEFT table

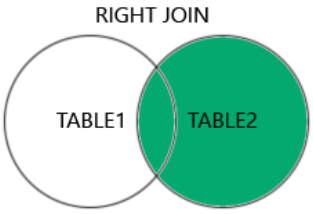


Table 1: Students (LEFT table)

student_id	name
1	Rahul
2	Priya
3	John

Table 2: Marks (RIGHT table)

student_id	marks
1	85
3	90
4	75

```
SELECT Students.student_id, Students.name, Marks.marks
FROM Students
RIGHT JOIN Marks
ON Students.student_id = Marks.student_id;
```

RESULT:

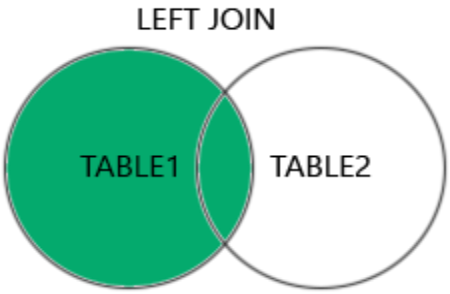
student_id	name	marks
1	Rahul	85
3	John	90
NULL	NULL	75

Student_id 4 → no matching student
So **NULL** is shown from Students table → name = NULL

LEFT JOIN:

LEFT JOIN returns:

All rows from the LEFT table
Matching rows from the RIGHT table
If no match is found → NULL is returned from the RIGHT table



The LEFT JOIN keyword returns all records from the left table (table1), and the matching records (if any) from the right table (table2).

STUDENTS TABLE

student_id	name
1	Rahul
2	Priya
3	John

MARKS TABLE

student_id	marks
1	85
3	90

Query:

```
SELECT Students.student_id, Students.name, Marks.marks
FROM Students
LEFT JOIN Marks
ON Students.student_id = Marks.student_id;
```

OUTPUT

student_id	name	marks
1	Rahul	85
2	Priya	NULL
3	John	90

Priya (id 2)
Not found in Marks table → shows **NULL**

CROSS JOIN:

CROSS JOIN gives the Cartesian product:
Every row from the first table **X** Every row from the second table
It does NOT use any condition (ON clause).
So output = All possible combinations.

COLORS:

color_id	color
1	Red
2	Blue

SIZES:

size_id	size
1	Small
2	Large

QUERY:

```
SELECT Colors.color, Sizes.size
FROM Colors
CROSS JOIN Sizes;
```

RESULT:

color	size
Red	Small
Red	Large
Blue	Small
Blue	Large

Explanation

- Red × Small
- Red × Large
- Blue × Small
- Blue × Large

Every color is paired with every size 2 × 2 = **4 rows**

SELF JOIN

A self join is a regular join, but the table is joined with itself.

SELF JOIN means:

A table is joined **with itself**
We use **different aliases** to treat the same table as two tables
Commonly used for **hierarchy** (manager → employee)

EMPLOYEE TABLE:

emp_id	name	manager_id
1	Rahul	NULL
2	Priya	1
3	John	1
4	Kumar	2

QUERY:(Employee–Manager)

```
SELECT
e.name AS Employee,
m.name AS Manager
FROM Employees e
LEFT JOIN Employees m
ON e.manager_id = m.emp_id;
```

RESULT:

Employee	Manager
Rahul	**NULL**
Priya	**Rahul**
John	**Rahul**
Kumar	**Priya**

Explanation
Priya → Manager = Rahul
Because Priya's manager_id = 1 (Rahul)
John → Manager = Rahul
Kumar → Manager = Priya
Rahul → Manager = NULL
He has no manager.

DATABASES

The CREATE DATABASE statement is used to create a new SQL database**.**

Syntax

CREATE DATABASE *databasename*;

Once a database is created, you can check it in the list of databases with the following SQL command:

SHOW DATABASES;

DROP DATABASE:

The DROP DATABASE statement is used to drop an existing SQL database.

Syntax

DROP DATABASE *databasename*;

Note: Be careful before dropping a database. Deleting a database will result in loss of complete information stored in the database!

CREATE TABLE :

The CREATE TABLE statement is used to create a new table in a database.

Syntax

```
CREATE TABLE table_name (
   column1 datatype,
   column2 datatype,
   column3 datatype,
  ....
);
```

EXAMPLE:

```
CREATE TABLE Persons (
   PersonID int,
   LastName varchar(255),
   FirstName varchar(255),
   Address varchar(255),
   City varchar(255)
);
```

Create Table Using Another Table

- A copy of an existing table can also be created using CREATE TABLE.
- The new table gets the same column definitions. All columns or specific columns can be selected.
- If you create a new table using an existing table, the new table will be filled with the existing values from the old table.

Syntax

```
CREATE TABLE new_table_name AS
    SELECT column1, column2,...
    FROM existing_table_name
    WHERE ....;
```

EMPLOYEE TABLE:

emp_id	name	dept	salary
1	Rahul	IT	30000
2	Priya	HR	32000
3	John	IT	28000
4	Kumar	Finance	45000

1)Create a New Table With Same Structure Only (No Data)

CREATE TABLE employees_copy AS
SELECT *
FROM employees
WHERE 1 = 0;

Table created: **employees_copy**
No rows copied

2)Create a New Table With All Data

CREATE TABLE employees_backup AS
SELECT *
FROM employees;

Copies **structure + all 4 rows**

3)Create a New Table With Selected Columns

Example: Copy only emp_id and name

```
CREATE TABLE emp_basic AS
SELECT emp_id, name
FROM employees;
```

Resulting table: emp_basic

emp_id	name
1	Rahul
2	Priya
3	John
4	Kumar

4)Create a New Table With Filtered Data

Example: Only IT department employees

```
CREATE TABLE employees_IT AS
SELECT *
FROM employees
WHERE dept = 'IT';
```

Resulting employees_IT table:

emp_id	**name**	**dept**	**salary**
1	Rahul	IT	30000
3	John	IT	28000

5)**Create a Table With Calculated Column**

Example: Add a bonus column (10% of salary)

```
CREATE TABLE salary_bonus AS
SELECT
   name,
   salary,
   salary * 0.10 AS bonus
FROM employees;
```

Resulting table: salary_bonus

name	**salary**	**bonus**
Rahul	30000	3000
Priya	32000	3200
John	28000	2800
Kumar	45000	4500

DROP TABLE:

The DROP TABLE statement is used to drop an existing table in a database.

Syntax

DROP TABLE *table_name*;

Note: Be careful before dropping a table. Deleting a table will result in loss of complete information stored in the table!

TRUNCATE TABLE

The TRUNCATE TABLE statement is used to delete the data inside a table, but not the table itself.

Syntax:

TRUNCATE TABLE *table_name*;

ALTER

Sample Table: employees
Let's assume we have the following table:

```
CREATE TABLE employees (
    emp_id INT,
    name VARCHAR(50),
    dept VARCHAR(30),
    salary INT
);
```

Sample data:

emp_id	name	dept	salary
1	Rahul	IT	30000
2	Priya	HR	32000

1)Add a New Column:

Example: Add age column

```
ALTER TABLE employees
ADD COLUMN age INT;
```

Explanation

This adds a new column **age** to the employees table.

After this, the table looks like:

emp_id	name	dept	salary	age
1	Rahul	IT	30000	NULL
2	Priya	HR	32000	NULL

2)Add Multiple Columns

```
ALTER TABLE employees
ADD COLUMN city VARCHAR(50),
ADD COLUMN join_date DATE;
```

Explanation

Adds two new columns (city and join_date) at once.

3)Modify Column Data Type

Example: Change salary to decimal format

```
ALTER TABLE employees
MODIFY COLUMN salary DECIMAL(10,2);
```

Explanation

The salary column now supports decimal values like 30000.50.

4)Rename a Column

Example: Rename name → full_name

```
ALTER TABLE employees
CHANGE COLUMN name full_name VARCHAR(50);
```

Explanation:

name column is renamed to full_name, with data type specified again.

5)Rename the Table

```
ALTER TABLE employees
RENAME TO staff;
```

Explanation

The table name changes from employees → staff.

6)Drop (Delete) a Column

Example: Remove city column

```
ALTER TABLE employees
DROP COLUMN city;
```

Explanation

This completely deletes the city column from the table.

7)Change Column Name + Data Type Together

```
ALTER TABLE employees
CHANGE COLUMN salary monthly_salary INT;
```

Explanation

- Renames salary → monthly_salary
- Changes type from DECIMAL to INT

MySQL Constraints

SQL constraints are used to specify rules for data in a table.

Constraints can be specified when the table is created with the CREATE TABLE statement, or after the table is created with the ALTER TABLE statement.

Syntax

```
CREATE TABLE table_name (
        column1 datatype constraint,
        column2 datatype constraint,
        column3 datatype constraint,
        ....
);
```

Constraints are rules applied to table columns to protect the correctness, accuracy, and reliability of data.

Common Constraints:

1. **PRIMARY KEY**
2. **FOREIGN KEY**
3. **UNIQUE**
4. **NOT NULL**
5. **CHECK**
6. **DEFAULT**

Below is each one with **simple examples**.

1)PRIMARY KEY

A **PRIMARY KEY** uniquely identifies each row in a table.
It **cannot be NULL** and **cannot repeat**.

EXAMPLE:

roll number in a school — every student has a unique one.

2)FOREIGN KEY

A **FOREIGN KEY** connects two tables.
It ensures that a value in one table **must exist** in another table.

3)UNIQUE

A **UNIQUE** constraint makes sure values in a column **are not duplicated**.
Two rows cannot have the same value in that column.

4)**NOT NULL**

A **NOT NULL** constraint means the column **must have a value**.
You cannot leave it empty.

Example:

An employee must have a name → name cannot be NULL.

5) CHECK

A **CHECK** constraint forces the value to follow a rule.

Examples:

- Age must be 18 or above
- Salary must be positive
- Quantity must be greater than 0

If the rule is broken → MySQL stops the insert/update.

6)**DEFAULT**

A **DEFAULT** constraint gives a column an automatic value when no value is provided.

Example:

If department is not given → set it automatically to "IT".

Constraint	What it Ensures
PRIMARY KEY	*Unique + Not Null*
FOREIGN KEY	*Valid relationship between tables*
UNIQUE	*No duplicate values*
NOT NULL	*Value must be provided*
CHECK	*Value must follow a rule*
DEFAULT	*Assigns a default value*

PRIMARY KEY:

1)Create Table with Primary Key

QUERY:

```
CREATE TABLE students (
student_id INT PRIMARY KEY,
name VARCHAR(50),
age INT
);
```

student_id uniquely identifies each student.

2)Create Table with AUTO_INCREMENT Primary Key

```
CREATE TABLE students (
   student_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50),
   age INT
);
```

student_id automatically gets values like 1, 2, 3, …
No need to manually insert it.

3) Add Primary Key to an Existing Table

If the table is already created:
Add single-column primary key:

```
ALTER TABLE students
ADD PRIMARY KEY (student_id);
```

4)Composite Primary Key (Multiple columns)

Used when **two columns together** make a unique row.

Example:
A student can attend many courses
But the same course cannot be assigned twice to the same student.

QUERY:

```
CREATE TABLE student_courses (
   student_id INT,
   course_id INT,
   PRIMARY KEY (student_id, course_id)
);
```

Combination (student_id, course_id) must be unique.

UNIQUE :

A UNIQUE constraint makes sure that no two rows have the same value in that column.

Duplicate values → Not allowed
NULL values → Allowed (but only one NULL in MySQL for UNIQUE)

It is used for things like email, phone number, username, etc.

1)Create Table with UNIQUE Constraint

```
CREATE TABLE users (
id INT PRIMARY KEY,
 name VARCHAR(50),
 email VARCHAR(50) UNIQUE
);
```

email must be different for every user
No duplicates allowed

2)Add UNIQUE to Existing Table

```
ALTER TABLE users
ADD UNIQUE (email);
```

3)UNIQUE on Multiple Columns (Composite Unique)

Example: A student can enroll only once in the same course.

```
CREATE TABLE student_courses (
   student_id INT,
   course_id INT,
   UNIQUE (student_id, course_id)
);
```

FOREIGN KEY:

A FOREIGN KEY links two tables together.
It ensures that the value in one table must exist in another table.

- Used to maintain relationship between tables
- Prevents invalid data

DEPARTMENT(PARENT)

dept_id	dept_name
1	HR
2	IT
3	Finance

EMPLOYEE(CHILD)

emp_id	name	dept_id
101	Rahul	1
102	Priya	2
103	John	3

dept_id in employees must match dept_id in departments

1)Create Table with FOREIGN KEY

First create the parent table:

```
CREATE TABLE departments (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50)
);
```

Then create the child table:

```
CREATE TABLE employees (
   emp_id INT PRIMARY KEY,
   name VARCHAR(50),
   dept_id INT,
   FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

- This ensures you can insert only valid dept_id
- No employee can have a department that does not exist

Example of Invalid Insert

```
INSERT INTO employees (emp_id, name, dept_id)
VALUES (104, 'Kumar', 10);
```

RESULT :INVALID
(Because dept_id **10 does NOT exist** in departments table.)

Example of Valid Insert

```
INSERT INTO employees (emp_id, name, dept_id)
VALUES (104, 'Kumar', 2);
```

Allowed because dept_id **2 exists** in departments.

TABLE

emp_id	name	dept_id
101	Rahul	1
102	Priya	2
103	John	3
104	Kumar	2

NOT NULL

NOT NULL means the column cannot be empty.
You must enter a value in that column.

SIMPLY NOT ALLOW NULL VALUE

QUERY:

```
CREATE TABLE students (
id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
age INT NOT NULL
);
```

name → cannot be NULL
age → cannot be NULL
You must enter both values when inserting.

Valid Insert

```
INSERT INTO students (id, name, age)
VALUES (1, 'Rahul', 20);
```

Invalid Insert (Error)

```
INSERT INTO students (id, name, age)
VALUES (2, NULL, 22);
```

CHECK

A CHECK constraint makes sure that the value you insert into a column must satisfy a condition.

Ensures data is valid

QUERY:

Example Table With CHECK

```
CREATE TABLE students (
   id INT PRIMARY KEY,
   name VARCHAR(50),
   age INT CHECK (age >= 18)
);
```

age must be **18 or above**
If age < 18 →error

Valid Insert

```
INSERT INTO students (id, name, age)
VALUES (1, 'Rahul', 20);
```

Because 20 ≥ 18

Invalid Insert (Error because CHECK fails)

```
INSERT INTO students (id, name, age)
VALUES (2, 'Kumar', 15);
```

Error: CHECK constraint failed
Because 15 is less than 18

More Examples of CHECK

1. **Salary must be positive**

   salary INT CHECK (salary > 0)

2. **Gender must be only 'M' or 'F'**
   gender CHAR(1) CHECK (gender IN ('M', 'F'))

3. **Marks between 0 and 100 marks**

   INT CHECK (marks BETWEEN 0 AND 100)

4. **Age must be between 18 and 60**

   age INT CHECK (age >= 18 AND age <= 60)

DEFAULT:

The DEFAULT constraint is used to set a default value for a column.

DEFAULT gives a predefined value to a column when you do NOT provide one during INSERT.

EXAMPLE:

```
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
name VARCHAR(50),
city VARCHAR(50) DEFAULT 'Mumbai'
);
```

If you don't enter the city → MySQL will use **'Mumbai'**.

Valid Insert WITHOUT City

```
INSERT INTO employees (emp_id, name)
VALUES (1, 'Rahul');
```

City will become **'Mumbai'** automatically

RESULT:

emp_id	name	city
1	Rahul	Mumbai

Insert WITH City

```
INSERT INTO employees (emp_id, name, city)
VALUES (2, 'Priya', 'Delhi');
```

RESULT:

Uses the value you provided

emp_id	name	city
1	Rahul	Mumbai
2	Priya	Delhi

Add DEFAULT to Existing Table

```
ALTER TABLE employees
ALTER city SET DEFAULT 'Mumbai';
```

MySQL CREATE INDEX:

An INDEX improves the speed of searching data in a table.

- Faster SELECT queries
- Works like the index pages in a book
- But slightly increases storage

1)Simple INDEX

Creates an index on a single column.

```
CREATE INDEX idx_name
ON employees(name);
```

EXPLAINATION:

- idx_name is index name
- Index created on name column
- Searching employees by name becomes faster

2. UNIQUE INDEX

Ensures values are unique (no duplicates).

```
CREATE UNIQUE INDEX idx_email
ON users(email);
```

3. Composite INDEX (Multiple Columns)

Used when WHERE uses multiple columns.

```
CREATE INDEX idx_name_city
ON employees(name, city);
```

Helps when searching using both name AND city.

5 Create Index during Table Creation

```
CREATE TABLE employees (
   id INT PRIMARY KEY,
   name VARCHAR(50),
   city VARCHAR(50),
   INDEX (name),
   INDEX (city)
);
```

6)Drop (Remove) an Index

```
DROP INDEX idx_name ON employees;
```

AUTO INCREMENT:

AUTO_INCREMENT automatically gives a new increasing number to a column when you insert a new record.

- No need to enter ID manually
- Automatically generates 1, 2, 3, 4, …
- Used mostly for PRIMARY KEY

Simple Example Table

```
CREATE TABLE students (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50)
);
```

Meaning:

- id starts from **1**
- Every new row gets next number automatically

Inserting Without ID

```
INSERT INTO students (name)
VALUES ('Rahul');
```

MySQL will set id = 1

```
INSERT INTO students (name)
VALUES ('Priya');
```

MySQL will set id = 2

Table After Inserts

id	name
1	Rahul
2	Priya

Set Custom Starting Value

Start auto-increment from 100:
ALTER TABLE students AUTO_INCREMENT = 100;
Next inserted row → id = 100

Add AUTO_INCREMENT to Existing Column

ALTER TABLE students
MODIFY id INT AUTO_INCREMENT;

Auto-increment column rules

- Only **one AUTO_INCREMENT** column per table
- Must be indexed (PRIMARY KEY or UNIQUE)
- Cannot be NULL

Insert With ID (Optional)

You *can* insert manually:

INSERT INTO students (id, name)
VALUES (10, 'John');

Then next auto-generated ID will be **11**.

VIEW

A VIEW in MySQL is a virtual table.

It is based on the result of a SELECT query.

✓ A view has rows and columns like a real table
✓ It does NOT store data
✓ It always shows up-to-date data

SYNTAX:

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

SAMPLE :

```
CREATE TABLE customers (
   customer_id INT PRIMARY KEY AUTO_INCREMENT,
   customer_name VARCHAR(50),
   country VARCHAR(50),
   city VARCHAR(50)
);
```

INSERT DATA:

```
INSERT INTO customers (customer_name, country, city) VALUES
('Rahul', 'India', 'Mumbai'),
('Priya', 'India', 'Delhi'),
('John',  'USA',   'New York'),
('Asha',  'India', 'Bangalore');
```

SAMPLE TABLE:

```
CREATE TABLE orders (
   order_id INT PRIMARY KEY AUTO_INCREMENT,
   customer_id INT,
   amount DECIMAL(10,2),
   order_date DATE,
   FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
)
```

INSERT DATA:

```
INSERT INTO orders (customer_id, amount, order_date) VALUES
(1, 2500.50, '2024-10-01'),
(2, 1500.00, '2024-10-05'),
(1, 3200.00, '2024-11-10'),
(4, 900.00,  '2024-11-15');
```

Create Views Using These Sample Tables

1. **View: Indian Customers Only**

```
CREATE VIEW indian_customers AS
SELECT customer_name, city
FROM customers
WHERE country = 'India';
```

Query:

```
SELECT * FROM indian_customers;
```

Output:

customer_name	city
Rahul	Mumbai
Priya	Delhi
Asha	Bangalore

2. **View: High Amount Orders (> 2000)**

```
CREATE VIEW high_value_orders AS
SELECT order_id, amount, order_date
FROM orders
WHERE amount > 2000;
```

Query:

```
SELECT * FROM high_value_orders;
```

Output:

order_id	amount	order_date
1	2500.50	2024-10-01
3	3200.00	2024-11-10

THANK YOU